Maximus Curkovic, 101139937

SYSC 4810 Assignment, Fall 2023

December 4th, 2023

Problem 1:

Part a)

Given the problem context, I believe that RBAC is the most suitable control model for the prototype. This is simply because there are numerous roles, and some roles are naturally going to have more permissions than others. For example, an Investment Analyst is going to have far more access to the system than a Regular Client.

Part b)

In the case of the access control representation, since we are using RBAC for the prototype, it only makes sense to utilize an access control matrix. This will easily depict each user and their permissions within the system and makes it easy for reference in the later sections, especially the password file. It will also be very simple to define users and roles for the policy, and assign permissions via parsing a CSV file.

Part c)

The below figure depicts the access control matrix for the Finvest Holdings security system's access control policy:

	Account Balance	Investments Portfolio	Contact Details	Money Market Instruments	Consumer Instruments	Derivatives Trading	Interest Instruments	Client Account	Access System Between 9AM-5PM	Validate Modifications
Regular Client	R	R	R (FA)	-	-	-	-	-	-	-
Teller	R	R	-	-	-	-	-	-	R, W	-
Financial Advisor	R	R, W	-	-	R	-	-	-	-	-
Compliance Officer	R	R	-	-	-	-	-	-	-	R, W
Premium Client	R	R, W	R (FA + IA)	-	-	-	-	-	-	-
Investment Analyst	R	R, W	-	R	R	R	R	-	-	-
Financial Planner	R	R, W	-	R	R	-	-	-	-	-
Technical Support	-	-	-	-	-	-	-	R, W	-	-

Figure 1: Access Control Matrix for the Finvest Holdings System

Part d)

For the Access Control Policy, three classes were created for its implementation. The source code can be find attached in the assignment submission.

- **AccessControlPolicy.py:** Defines a class for the AccessControlPolicy. The constructor defines an inputted role, which is what is passed when the CSV file is read. I also have added getters (getRole) and setters (setRole) as well if necessary. getPermissions() reads through the attached AccessControlMatrix.csv file (which stores the Access Control Matrix created in part c) and simply reads through lines and parses the data for the role that was specified in the policy. If the role is found, it is the "right" data and is set as the role of the user.

```python
import csv


class AccessControlPolicy:
    def __init__(self, role) -> None:
        self._role = role

    def setRole(self, set_role) -> str:
        self._role = set_role

    def getRole(self) -> str:
        return self._role

    def getPermissions(self) -> dict:
        with open("AccessControlMatrix.csv", "r") as CSVFile:
            CSVReader = csv.DictReader(CSVFile)
            data = [row for row in CSVReader]
            for item in data:
                found_role = item.pop("Role")
                if found_role == self._role:
                    return item
```

Figure 2: AccessControlPolicy class with its getters and setters

- **User.py**: Defines a class for the User. A User class instance has a username (which is important for the remainder of the assignment), a name, and a role. I provided getters and setters for both username and name if needed. getRole() gets the role provided from the AccessControlPolicy and is used to define a particular User. getPermissions() gets all of the permissions that are assigned to a User, in Dictionary form, where the key is the permission and the value is the access level (R or RW).

```python
from AccessControlPolicy import AccessControlPolicy


class User:
    def __init__(self, username, name, role):
        self.__username = username
        self.__name = name
        self.__role = role

    def getName(self):
        return self.__name

    def getUsername(self):
        return self.__username

    def setName(self, name):
        self.__name = name;

    def setUsername(self, username):
        self.__username = username;

    def getRole(self) -> AccessControlPolicy:
        return self.__role.getRole()

    def getPermissions(self) -> dict:
        return self.__role.getPermissions()
```

Figure 3: User class with its getters and setters

- **UserList.py:** There are multiple parts to this file. The first part defines a policy, and we instantiate all the roles used by Finvest Holdings within the policy.

```python
import User
import AccessControlPolicy

policy = AccessControlPolicy

regularClientRole = policy.AccessControlPolicy("Regular Client")
premiumClientRole = policy.AccessControlPolicy("Regular Client")
financialAdvisorRole = policy.AccessControlPolicy("Financial Advisor")
financialPlannerRole = policy.AccessControlPolicy("Financial Planner")
investmentAnalystRole = policy.AccessControlPolicy("Investment Analyst")
complianceOfficerRole = policy.AccessControlPolicy("Compliance Officer")
technicalSupportRole = policy.AccessControlPolicy("Technical Support")
tellerRole = policy.AccessControlPolicy("Teller")
```

Figure 4: UserList file that defines the different roles for the Access Control Policy

The second part of the class is a method which instantiates all of the users for the Finvest Holdings system, and assigns them roles based on what was defined above. A list of all the users is returned.

```
def instantiateUsers():

    mischa = User.User("mischa", "Mischa Lowery", regularClientRole)
    veronica = User.User("veronica", "Veronica Perez", regularClientRole)

    winston = User.User("winston", "Winston Callahan", tellerRole)
    kelan = User.User("kelan", "Kelan Gough", tellerRole)

    nelson = User.User("nelson", "Nelson Wilkins", financialAdvisorRole)
    kelsie = User.User("kelsie", "Kelsie Chang", financialAdvisorRole)

    howard = User.User("howard", "Howard Linkler", complianceOfficerRole)
    stefania = User.User("stefania", "Stefania Smart", complianceOfficerRole)

    willow = User.User("willow", "Willow Garza", premiumClientRole)
    nala = User.User("nala", "Nala Preston", premiumClientRole)

    stacy = User.User("stacy", "Stacy Kent", investmentAnalystRole)
    keikilana = User.User("keikilana", "Keikilana Kapahu", investmentAnalystRole)

    kodi = User.User("kodi", "Kodi Matthews", financialPlannerRole)
    malikah = User.User("malikah", "Malikah Wu", financialPlannerRole)

    caroline = User.User("caroline", "Caroline Lopez", technicalSupportRole)
    pawel = User.User("pawel", "Pawel Barclay", technicalSupportRole)

    userList = [mischa, veronica, winston, kelan, nelson, kelsie, howard,
                stefania, willow, nala, stacy, keikilana, kodi, malikah, caroline, pawel]

    return userList
```

Figure 5: instantiateUsers() method in UserList.py that assigns roles to the users

Part e)

Testing the access control policy is very straightforward, and scales for when new users are added, either in the instantiateUsers() method or in the CSV file where the Access Control matrix is stored. In the UserList.py file, there is a main function that instantiates the users, and reads for each user in the list, assigns their permissions, and prints out the results for each user.

```
def main():
    for user in instantiateUsers():
        print(user.getName() + ", role: " + user.getRole() + ", permissions: ")
        permissions = user.getPermissions()
        for permission in permissions:
            print(permission + "=" + permissions[permission])
        print("\n")

main()
```

Figure 6: main() method that prints all permissions for registered users

The tests work perfectly. Below is an example user that is printed. When the test runs, every user is printed (which leads to 16 blocks of prints) but below is an excerpt taken from the long print. As we see, the user name is printed, along with their role and their permissions match those in the matrix.

```
Mischa Lowery, role: Regular Client, permissions:
Account Balance=R
Investments Portfolio=R
Financial Advisor Contact Details=R
Investment Analyst Contact Details=
Money Market Instruments=
Consumer Instruments=
Derivatives Trading=
Interest Instruments=
Client Account=
Access Between 9AM-5PM=
Validate Modifications=
```

Figure 7: Example of printed permissions for Mischa Lowery

Problem 2:

Part a)

For this assignment, I will be using SHA-256, which can be found in the hashlib library provided in Python. This hash function allows you to specify the specific length of the hash that you want, as well as a generated salt to add. I will be using SHA-256 over the other options for two reasons: it is one of the fastest compared to others like SHA-512, and SHA-256 has no known security vulnerabilities that make it insecure and has not been “broken”. I believe that this hash function is quite simple to implement for hashing passwords, while still providing a reasonable level of security.

Part b)

The password file will contain three things. Firstly, each user will be identified and have their own associated “id” that uniquely identifies them within the file. For each user, the password file will store two separate values: a **salt**, which is added to the password hash for an extra layer of security, and **the password hash itself**. The Password encrypter takes in a password parameter, stores a hash of it (using SHA-256, as mentioned in part a) and writes it to the file for the associated user.

Part c) The implementation of the password file consists of two files:

- **PasswordEncrypter.py**: This is, overall, where most of the functionality for the password file generator is. We define a class, which has one attribute (simply the created password file – **passwd.txt**). encryptPassword takes in a username and password, creates a salt using 32 random bits, adds the salt to the password, and then hashes the whole password. Afterwards, the password is written to the file along with the inputted username. retrievePassword takes in a username, and will return that associated password from the file.

```python
import hashlib
import random

1 usage
class PasswordEncrypter:
    def __init__(self) -> None:
        self.__passwordFile = "./passwd.txt"

    16 usages
    def encryptPassword(self, username, password):
        file = open(self.__passwordFile, "a")
        salt = str(random.getrandbits(32))
        password += salt
        passwordHash = hashlib.sha256(password.encode())
        file.write(username + " " + salt + " " + passwordHash.hexdigest() + "\n")
        file.close()
        return

    def retrievePassword(self, username) -> str:
        file = open(self.__passwordFile, "r")
        for line in file:
            line = line.split(" ")
            if line[0] == username:
                return line[2]
        return ""
```

Figure 8: PasswordEncrypter class, depicting the constructor, encryptPassword and retrievePassword functions

- **UserList.py (Updated)**: For each user in instantiateUsers, it now also encrypts their passwords into the passwd.txt file. Note that the code excerpt below only shows a select group of users; there are 10 more that also had their passwords encrypted as well.

```
def instantiateUsers():

    mischa = User.User( username: "mischa",  name: "Mischa Lowery", regularClientRole)
    encrypter.encryptPassword( username: "mischa",  password: "Grapefruit5$")
    veronica = User.User( username: "veronica",  name: "Veronica Perez", regularClientRole)
    encrypter.encryptPassword( username: "veronica",  password: "Grapefruit5$")

    winston = User.User( username: "winston",  name: "Winston Callahan", tellerRole)
    encrypter.encryptPassword( username: "winston",  password: "Grapefruit5$")
    kelan = User.User( username: "kelan",  name: "Kelan Gough", tellerRole)
    encrypter.encryptPassword( username: "kelan",  password: "Grapefruit5$")

    nelson = User.User( username: "nelson",  name: "Nelson Wilkins", financialAdvisorRole)
    encrypter.encryptPassword( username: "nelson",  password: "Grapefruit5$")
    kelsie = User.User( username: "kelsie",  name: "Kelsie Chang", financialAdvisorRole)
    encrypter.encryptPassword( username: "kelsie",  password: "Grapefruit5$")
```

Figure 9: instantiateUsers(), now with the passwords encrypted

Part d) When all of the user passwords are encrypted (i.e. the UserList.py file is ran), the following **passwd.txt** file is created:

```
mischa 297945006 25e4ab6bf17bec2031c491c0e16f6bfcbfd134483ef9834764ffda4b3dfe9427
veronica 3924854415 ad65dd01e2927f4e4b5a2a7c670f59c8c8f295947487c36ef130b32f6698372c
winston 3623968142 2024fc46d1bd65b6a6d71c63a0335c5c6e9cbeb6edd042f47fa3d813c7ec8d5d
kelan 1293198070 33bab08360ac5994c34c6837aed4ceca878b99078f1f4e9b0641b0db0f7224b1
nelson 3673780222 4eab0878d8d1c30286e6bbd26acc40242b2b864aabdbe7e7f61b8de2de6b5b7b
kelsie 274114821 d7ac030ce95def8d16e85fc0981cc1f79704e6cd0bacefca2bc3cfed854b8e6a
howard 3097980108 cb6af5e797c7d3c9b5035f2354c90b0db503d440b83daa59720b6aa2745656ca
stefania 1062295288 92e4b0975dedac07c2d9e0a2b65fc422e5b9da8765454a3e0a635b4aef4d2baf
willow 2113509260 f9ce6911b1c1ed4ead36c33b5b8572c737fa9cd47e0a0d70141e2c67bbcfb647
nala 1940533163 ed2fab3062b44add90248a88da404862762a177f5805ac3cf85badfd373d6385
stacy 4061185599 58831a6eaee36612e0031f1f78f232c6eea72f6408dd2e166ec948dbb957a7e2
keikilana 3175754316 c4e83d960e5aeada3e8dbfa2f3d1d056eebf31714d6d1c7b87201b7a2aafe447
kodi 3819257614 bfdd35926e62d5d16e24f1af0e549cc18d38a040ec9ae23daac1f430ca18ff6c
malikah 2174668381 54e8809025e85ae8b51b442444b917d1cfcfc66db71c0ba02d288c30c0cfd244
caroline 2497857735 db6e985fd0fcc1b1b69a0236aa7785ea359ae6a9463f717afd7151e18a875081
pawel 2929122015 5d99124b584f69e1d1374239b034a2bc5c3f6f1dbc712995dcf1915ed4b3f375
```

Figure 10: passwd.txt file

This shows that adding the user passwords was successful. As we see in the file, we have each user and their associated salt and hashed password.

Problem 3:

Part a) The figure below shows the general User Interface design for the Finvest Holdings system. There are two things that a user can do: they can register an account (entering a username + password that does <u>not</u> exist within the system), or login to an account (more on this in Problem 4). Note that the password input will be hidden using a Python library called **getpass**, which hides the user input.

```
Finvest Holdings: Client Records System
Enter L to login, or R to register a new account:
```

Figure 11: Introductory interface to the Finvest Holdings system

When enrolling an account, there are two outcomes. If the user registers with a password <u>that passes the password checker</u> (more on this in part b), then a "Registration successful" message will pop up, and they will be redirected back to the main screen.

```
Finvest Holdings: Client Records System
Register User
First, enter a valid role. List of valid roles include:
Regular Client
Premium Client
Financial Advisor
Financial Planner
Investment Analyst
Compliance Officer
Technical Support
Teller
Role: Teller
Next, enter a username and password.
Username: maxicurk
Enter password:
ddasdasda5$F
Account successfully created!
```

Figure 12: Example of a user with username maxicurk having an account successfully created

If they do not properly enter a password that <u>passes the password checker</u> (again, more on this in part b), then a "Registration unsuccessful" message will pop up, show the password criteria required, and will be redirected back to the main screen to try again.

```
Next, enter a username and password.
Username: maxicurky
Enter password:
g
Account creation unsuccessful. Your password must adhere to the following criteria:
   - 8 to 12 characters in length
   - Include one upper-case letter, one lower-case letter, number and special character from {!, @, #, $,%, ?, *
   - Not be a weak password as determined by a list of common passwords
   - Not be a calendar date, license plate number, telephone number, or other common number
   - Not match your username
```

Figure 13: Example of a user with username maxicurky not having a strong enough password

Lastly, if the user enters a username that already exists within the system, this will be caught and will be printed to the user. A “Registration unsuccessful” message will pop up, and the user will be redirected back to the main screen to try again.

```
Next, enter a username and password.
Username: maxicurk
Account creation unsuccessful. You are trying to enter a username that already exists.
```

Figure 14: Example of a user with username maxicurk trying to use a username that already exists in passwd.txt

Part b) The password checker should be one singular file, as it will contain a lot of methods that will be ran on an input password to ensure that it passes the criterion.

The 8-12 characters in length is quite easy to check, as one simply must iterate through each character of the password and check if it is in the range.

The individual characters can also be done in a very similar manner, by checking each individual character, to see if it matches and set a corresponding boolean value to True. If each value (upper-case, lower-case, etc.) is True, then the password passes that criterion.

A “weak password” is defined as a password used very commonly. A list of these common passwords will be created (I will follow [30 most common passwords of 2023 (beckershospitalreview.com)] for this, as well as the ones listed in the assignment sheet) and check whether the input password matches any of the listed common passwords.

Calendar dates, license plate numbers, telephone numbers, or other common numbers will each be a bit more difficult to implement and will each be their own method with their own functionality. The best way to check would be to see if a calendar date is 8 numerical digits, and a telephone number is 9 numerical digits. We can check for a license plate if the first 4 digits are letters, and the last 3 are numbers. I am under the assumption that we are checking Ontario license plate format.

Finally, the password checker will check that the input password is not equal to the user’s username in a separate method.

In the User Interface, when a user is prompted for a password and enters such password into the system, all of these conditions will be ran and must pass in order for the check to pass and the password to be validated.

Part c) For this part, I will separate between two types of files: new files and changed files (files that existed before that required refactoring for this problem).

New Files:

- **UserInterface.py:** This file defines the user interface for the Finvest Holdings system. The first method, userInterface() implements the general interface. The user is prompted to either click "L" or "l" to login, and "R" or "r" to register. The user is prompted to re-enter a command if it is anything other than the valid commands.

```
import getpass
import PasswordChecker
import PasswordEncrypter

passwordManager = PasswordEncrypter.PasswordEncrypter()
check = PasswordChecker

listRoles = ["Regular Client", "Premium Client", "Financial Advisor", "Financial Planner", "Investment Analyst",
             "Compliance Officer", "Technical Support", "Teller"]

3 usages
def userInterface() -> None:
    print("\nFinvest Holdings: Client Records System")
    registerOrLogin = input("Enter L to login, or R to register a new account: ")
    if registerOrLogin == "L" or registerOrLogin == "l":
        print("Entering login... ")
        loginUI()
    elif registerOrLogin == "R" or registerOrLogin == "r":
        print("Entering register... ")
        registerUI()
    else:
        print("Invalid command. Type L or R. Please try again")
        userInterface()
```

Figure 15: UserInterface.py, showing the userInterface() function

Next, there is a new registerUI() method. This method defines the entire interface for registering a user into the Finvest Holdings system. Once a user enters the interface, they are prompted to enter a valid role. The list of valid roles is printed out for the user. This role is added to the passwd.txt file (see the "changed files" section) for the purpose of defining a user role and retrieving it in order to print out their permissions when logging in (more on this in Problem 4).

```
def registerUI():
    print("\nFinvest Holdings: Client Records System")
    print("Register User")
    print("First, enter a valid role. List of valid roles include: ")
    for item in listRoles:
        print(item)
    role = input("Role: ")
    if role not in listRoles:
        print("Account creation unsuccessful. Role not valid.")
        registerUI()
```

Figure 16: registerUI() function showing the role validation

Next, the user is prompted to enter a username. The system will reject the username if it is already found in the passwd.txt file, and the user will be redirected to try again.

```
print("Next, enter a username and password.")
username = input("Username: ")
if passwordManager.checkExistingUsername(username):
    print("Account creation unsuccessful. You are trying to enter a username that already exists. ")
    registerUI()
```

Figure 17: registerUI() function showing the username validation

When the username check passes, the user is prompted to enter a password. This password will be checked to see if it matches the conditions listed in the password checker. If it does, the account is successfully created. Otherwise, the user's password will be rejected, a list of criteria for the password is shown, and the user is redirected to try again. The password is secured using Python's **getpass** library – this works very similarly to Linux machines, as the user will not be able to see the password they are entering. This adds an extra layer of security to the feature.

```
password = getpass.getpass(prompt="Enter password: ")
print(password)
if check.checkIfUsername(username, password) == False and check.checkLicensePlate(
        password) == False and check.checkCalendarDateOrPhoneNumber(password) == False and check.checkIfWeak(password) == False and check.checkCharacters(
        password) == True and check.checkPasswordLength(password) == True:
    passwordManager.registerAccount(username, role, password)
    print("Account successfully created!")
    userInterface()
else:
    print("Account creation unsuccessful. Your password must adhere to the following criteria: ")
    print("   - 8 to 12 characters in length")
    print(
        "   - Include one upper-case letter, one lower-case letter, number and special character from {!, @, #, $,"
        "%, ?, *")
    print("   - Not be a weak password as determined by a list of common passwords")
    print("   - Not be a calendar date, license plate number, telephone number, or other common number")
    print("   - Not match your username")
    registerUI()
```

Figure 18: registerUI() function showing the password validation

Lastly, the file contains a main() method that must be ran in order to launch the system.

```
def main():
    userInterface()


main()
```

Figure 19: main() function to run the userInterface() method

- **PasswordChecker.py:** This file implements the checks on the password, as illustrated in the assignment sheet. It does not have to be a class, as the program is only concerned with using the defined methods.

  Firstly, there is a function checkPasswordLength(password) that checks if the password is at least 8 characters. It iterates through the inputted string, counts all of the characters, and returns True if the password passes this check.

```
def checkPasswordLength(password) -> bool:
    counter = 0
    for char in password:
        counter += 1
    if counter < 8:
        print("For testing purposes: Password is not at least 8 characters long!")
        return False
    return True
```

Figure 20: checkPasswordLength() function

To check that the password contains at least one upper-case character, one lower-case character, one number, and one special digit defined from ["!", "@", "#", "$", "%", "?", "*"], a singular function called checkCharacters(password) does all of this functionality at once using boolean expressions. Python provides built-in Char functions that check if a character is numeric, upper-case, and lower-case, and these functions are used for such purposes. The special character is simply checked if the password string contains a character from the list of provided special characters. If all of these conditions are true, then the password passes this check.

```
def checkCharacters(password) -> bool:
    containsUpper = False
    containsLower = False
    containsNumber = False
    containsSpecial = False
    specialCharacters = ["!", "@", "#", "$", "%", "?", "*"]
    for char in password:
        if char.isupper():
            containsUpper = True
        elif char.islower():
            containsLower = True
        elif char.isnumeric():
            containsNumber = True
        elif char in specialCharacters:
            containsSpecial = True
    if containsNumber and containsUpper and containsLower and containsSpecial:
        return True
    print("For testing purposes: Does not contain either an upper case, lower case, number, or special character!")
    return False
```

Figure 21: checkCharacters() function

To check if a password is a “weak” or common password, I used a website to search for the most common passwords used (as can be seen in the comment). The method checkIfWeak(password) simply checks if the inputted password matches any of the passwords in the list, and passes the check if it does not.

```
# For this one, using the passwords from the following website: https://www.beckershospitalreview.com/cybersecurity/30-most-common-passwords-of-2023.html
1 usage
def checkIfWeak(password) -> bool:
    weakPasswordList = ["123456", "password", "123456789", "12345", "12345678", "qwerty", "1234567", "111111",
                        "1234567890", "123123", "abc123", "1234", "password1", "iloveyou", "1q2w3e4r", "000000",
                        "qwerty123", "zaq12wsx", "dragon", "sunshine", "princess", "letmein", "654321", "monkey",
                        "27653", "1qaz2wsx", "123321", "qwertyuiop", "superman", "asdfghjkl"]
    if password in weakPasswordList:
        print("For testing purposes: password is considered a common and weak password!")
        return True
    return False
```

Figure 22: checkIfWeak() function

To check if the password is the user’s username, the function checkIfUsername(username, password) simply compares the two inputted strings and makes sure they are not equal. If they are not equal, the check passes.

```
def checkIfUsername(username, password):
    if username == password:
        print("For testing purposes: password is your username!")
        return True
    return False
```

Figure 23: checkIfUsername function

To check for a calendar date or phone number, the function checkCalendarDateOrPhoneNumber(password) checks if there are at least 6 numerical values found anywhere within the inputted password. If there are, the password will be assumed as a calendar date or phone number, and is rejected. Otherwise, the check passes.

```
def checkCalendarDateOrPhoneNumber(password) -> bool:
    counter = 0
    curr = 0
    prev = 0
    for char in password:
        if char.isnumeric():
            counter += 1
        elif password[curr].isnumeric() and password[prev].isnumeric() and counter > 0:
            counter += 1
        elif counter != 6:
            counter = 0
        if counter >= 6:
            print("For testing purposes: Password may be a calendar date or phone number!")
            return True
        curr += 1
        prev = curr - 1
    return False
```

Figure 24: checkCalendarDateOrPhoneNumber() function

The last check that is completed is a license plate check. The function checkLicensePlate(password) checks if the password matches an Ontario license plate format (example: ABCD123), and will be rejected even if it passes the checkCharacters(password) check. Otherwise, the check passes.

```
# Check if a license plate - we are assuming Ontario format. This function tests for 7 characters - 4 letters, 3 numbers in order
1 usage
def checkLicensePlate(password) -> bool:
    if len(password) >= 7:
        if password[0].isalpha() and password[1].isalpha() and password[2].isalpha() and password[3].isalpha():
            if password[4].isdigit() and password[5].isdigit() and password[6].isdigit():
                print("For testing purposes: Password may be a license plate value!")
                return True
    return False
```

Figure 25: checkLicensePlate() function

Changed Files:

- **Passwd.txt:** The only change that was made compared to Problem 2 is the addition of a role column. This is mainly used for logging a user in (see Problem 4).

```
mischa Regular Client 2606918757 1b72f2f1bdae4ca96b59f555229323e31c95be37cf923b1acd4390bd91c87d22
```

Figure 26: Example of entry in passwd.txt with the added role

- **PasswordEncrypter.py:** There are two primary changes that were made in this problem – one new function, and one change to an existing function. The first addition is a function checkExistingUsername(self, username) function, which reads through the passwd.txt file and compares the username input to the usernames found within the file. If one matches, the method returns True, and False otherwise. This is used in the User Interface when a user types in a username.

```
1 usage
def checkExistingUsername(self, username):
    file = open(self.__passwordFile, "r")
    for account in file:
        account = account.split(" ")
        if account[0] == username:
            file.close()
            return True
    file.close()
    return False
```

Figure 27: checkExistingUsername() function

The second addition is a change to the original encryptPassword(self, username, password) function. This function, now called registerAccount(self, username, role, password) accounts for the role change that was mentioned previously.

```python
def registerAccount(self, username, role, password):
    file = open(self.__passwordFile, "a")
    salt = str(random.getrandbits(32))
    password += salt
    passwordHash = hashlib.sha256(password.encode())
    file.write(username + " " + role + " " + salt + " " + passwordHash.hexdigest() + "\n")
    file.close()
    return
```

Figure 28: registerAccount() function with the added role parameter

Part d) To test the new enroll user and password checker functionality, a new user was created using the password “Grapefruit5$”. This password was chosen specifically, as it passes all the requirements. The below figures walk through this process:

```
Finvest Holdings: Client Records System
Enter L to login, or R to register a new account: R
Entering register...
```

Figure 29: Finvest Holdings General interface, entering the Register interface

```
Finvest Holdings: Client Records System
Register User
First, enter a valid role. List of valid roles include:
Regular Client
Premium Client
Financial Advisor
Financial Planner
Investment Analyst
Compliance Officer
Technical Support
Teller
Role: Teller
```

Figure 30: Example of entering a role into the Register interface

```
Next, enter a username and password.
Username: maxcurkovic
Enter password:
Grapefruit5$
Account successfully created!
```

Figure 31: Example of a user with username “maxcurkovic” and password “Grapefruit5$” being created

Note that “Grapefruit5$” is not shown to the user– this is only printed for test purposes.

```
caroline Technical Support 1023309634 804dc5379c2f38f58a0184726713e15f874f07b96211d333882c035e74cdccf7
pawel Technical Support 2412850784 299c37238fce3cc15286ee6627ff347de74fa875c9bed86b7f9d46b20ac4e9e2
maxi Teller 175374228 9691112828dfc361b2251a6039a7a06317160288c305347c98c29b82c36c0023
maxcurkovic Teller 2681714053 09a5b76f774c2ea67646f3175c171ce8faad4b8c743776496e8ba794b49c1d77
```

Figure 32: Same user being added to the passwd.txt file

To test the password checker, a variety of passwords are fed as input into the checker, and the reason for failure is printed.

```
Next, enter a username and password.
Username: maximus
Enter password:
grape
For testing purposes: Does not contain either an upper case, lower case, number, or special character!
Account creation unsuccessful. Your password must adhere to the following criteria:
```

Figure 33: Example of a failed password “grape”

```
Next, enter a username and password.
Username: maximus
Enter password:
Grape5$
For testing purposes: Password is not at least 8 characters long!
Account creation unsuccessful. Your password must adhere to the following criteria:
```

Figure 34: Example of a failed password “Grape5$”

```
Next, enter a username and password.
Username: maximus
Enter password:
maximus
For testing purposes: password is your username!
Account creation unsuccessful. Your password must adhere to the following criteria:
```

Figure 35: Example of a failed password “maximus”

```
Next, enter a username and password.
Username: maximus
Enter password:
password
For testing purposes: password is considered a common and weak password!
Account creation unsuccessful. Your password must adhere to the following criteria:
```

Figure 36: Example of a failed password “password”

```
Next, enter a username and password.
Username: maximus
Enter password:
Grapefruit5$6131231245
For testing purposes: Password may be a calendar date or phone number!
Account creation unsuccessful. Your password must adhere to the following criteria:
```

Figure 37: Example of a failed password “Grapefruit5$6131231245”

```
Next, enter a username and password.
Username: maximus
Enter password:
Grapefruit5$20231130
For testing purposes: Password may be a calendar date or phone number!
Account creation unsuccessful. Your password must adhere to the following criteria:
```

Figure 38: Example of a failed password “Grapefruit5$6131231245”

```
Next, enter a username and password.
Username: maximus
Enter password:
aBcD123%
For testing purposes: Password may be a license plate value!
Account creation unsuccessful. Your password must adhere to the following criteria:
```

Figure 39: Example of a failed password “aBcD123%”

As can be seen above, the password checker works very well for its intended use and testing the password criteria.

Problem 4:

Part a) Recall the general User Interface design for the Finvest Holdings system. There are two things that a user can do: they can register an account (see problem 3), or login to an account (if a username exists and the password matches).

```
Finvest Holdings: Client Records System
Enter L to login, or R to register a new account: l
Entering login...
```

Figure 40: Finvest Holdings General interface, entering the Login interface

If a User inputs a username that exists, and the password associated with the username matches what is stored in the password file, then a "login successful" message pops up and the user will be able to access the system.

```
Finvest Holdings: Client Records System
Login User
Enter your username: mischa
Enter your password:

Access Granted. Login successful
```

Figure 41: Example of a "login successful" interface

If the password is not correct, a "login failed" message appears and the user will be redirected to try again.

```
Finvest Holdings: Client Records System
Login User
Enter your username: mischa
Enter your password:
Login failed. Your username or password is incorrect. Please try again
```

Figure 42: Example of a "login failed" interface

Part b) There were no new files required to implement the password verification mechanism. A new function was added to PasswordEncrypter.py, loginAccount(self, username, password) to login a user and verify their username and password.

Firstly, the passwd.txt file is opened, like registering a user. The registered user is split into its own "table" – the index refers to the column (0 is the username, 1 is the role, 2 is the salt, 3 is the hashed password).

Once the user is found within the file via the inputted username, the user's hashed password is replaced by a brand new hashed password in the file, with the **same salt added to the password.** This is very important, as the only way to compare the passwords is through the salt.

If the hashed passwords are compared and are equal because of the salt, there are a tuple of two values returned: one being True (indicating that the user credentials are validated), and the other being the role that is used for finding the user's permissions in the interface. If the username does not exist in the file, or the user does not enter the correct password for the right username, then the login fails.

```
def loginAccount(self, username, password) -> (bool,str):
    file = open(self.__passwordFile, "r")
    for user in file:
        user = user.split(" ")
        user[3] = user[3].replace( __old: "\n",  __new: "")
        if user[0] == username:
            password = password + user[2]
            passwordHash = hashlib.sha256(password.encode())
            if passwordHash.hexdigest() == user[3]:
                file.close()
                return True, user[1]
    file.close()
    return False, None
```

Figure 43: loginAccount() method to validate a user's login credentials

Part c) The login interface, found under the function loginUI() in UserInterface.py, is responsible for displaying the inputs for the user's username and password and for displaying the user's username, role, and permissions once the user's credentials are validated.

When accessed, the user is prompted for their username and associated password. The password also utilizes Python's **getpass** library, which will hide the user's typed password, similarly to Linux.

```
def loginUI():
    print("\nFinvest Holdings: Client Records System")
    print("Login User")
    username = input("Enter your username: ")
    password = getpass.getpass(prompt="Enter your password: ")
```

Figure 44: loginUI() function within UserInterface.py, depicting username and password inputs

Once the user's credentials are inputted, they will undergo a login check, completed by the loginAccount method in PasswordEncrypter. The user's inputted username and password is fed into the function, and it returns a tuple value of isLoggedIn and the user's role. The user's role is then defined from the Access Control Policy, and a new User entity can be created in order to access their permissions.

If isLoggedIn remains a value of True, the user will receive an "access granted: logged in" message, display their username and role within the system, and list the permissions that they have. To maintain flow of the program, a feature was implemented where the user can either logout and return to the main user interface, or quit the program using any key. If isLoggedIn is set to False, then the user will receive a "login failed" message and be redirected back to the main login page to try again.

```
(isLoggedIn, role) = passwordManager.loginAccount(username, password)
userRole = AccessControlPolicy.AccessControlPolicy(role)
newUser = User.User(username, userRole)
if isLoggedIn:
    print("\nAccess Granted. Login successful\n")
    print("Username: " + username)
    print("Role: " + role)
    print("Permissions: ")
    userPermissions = newUser.getPermissions()
    for permission in userPermissions:
        print(permission + "=" + userPermissions[permission])
    print("\n")
    command = input("Type L to logout, any other key to quit program: ")
    if command == "L" or "l":
        userInterface()
    else:
        exit()
else:
    print("Login failed. Your username or password is incorrect. Please try again")
    loginUI()
```

Figure 45: Remainder of loginUI(), depicting login successful/unsuccessful actions

Part d) Recall the user that was created for testing in Problem 3d, in Figure 32.

To test the user login and access control enforcement, this user will be utilized again. Once the login UI is accessed, the user's credentials are inputted. Note that the password is only shown here for testing purposes, and the user will not see it printed in the system.

```
Finvest Holdings: Client Records System
Login User
Enter your username: maxcurkovic
Enter your password:
Grapefruit5$
```

Figure 46: Example of a user with username "maxcurkovic" and password "Grapefruit5$" being logged in

Since these are the correct credentials, the login is successful, and the user's username, role, and permissions are shown. The user's role is "Teller", and verifying with the Access Control matrix, these permissions are displayed correctly.

```
Access Granted. Login successful

Username: maxcurkovic
Role: Teller
Permissions:
Account Balance=R
Investments Portfolio=R
Financial Advisor Contact Details=
Investment Analyst Contact Details=
Money Market Instruments=
Consumer Instruments=
Derivatives Trading=
Interest Instruments=
Client Account=
Access Between 9AM-5PM=RW
Validate Modifications=


Type L to logout, any other key to quit program:
```

Figure 47: Example of a user with username "maxcurkovic" and role "Teller" with their permissions

In the case that the user does not input the username or password correctly, it is caught by the system. As seen below, the correct username "maxcurkovic" is inputted, but the password is not "Grapefruit5$".

```
Finvest Holdings: Client Records System
Login User
Enter your username: maxcurkovic
Enter your password:
IncorrectPassword
Login failed. Your username or password is incorrect. Please try again
```

Figure 48: Example of a user with username “maxcurkovic” inputting incorrect password

From the figures above, it can be concluded that the login user mechanism and the access control enforcement are successfully implemented.

Problem 5:

This section summarizes the Finvest Holdings system and addresses the detailed contractual obligations, taking the client's requirements and constraints into consideration.

To start, the system contains the following files that are necessary for the system's operation:

.idea	11/30/2023 6:52 PM	File folder	
__pycache__	11/30/2023 6:09 PM	File folder	
AccessControlMatrix	11/30/2023 3:23 PM	Microsoft Excel Com...	1 KB
AccessControlPolicy.py	11/26/2023 9:28 PM	Python.File	1 KB
passwd	11/30/2023 6:51 PM	Text Document	2 KB
PasswordChecker.py	11/30/2023 12:06 PM	Python.File	4 KB
PasswordEncrypter.py	11/30/2023 5:01 PM	Python.File	2 KB
User.py	11/30/2023 3:47 PM	Python.File	1 KB
UserInterface.py	11/30/2023 6:10 PM	Python.File	4 KB
UserList.py	11/30/2023 6:51 PM	Python.File	4 KB

Figure 49: Directory and list of files for the Finvest Holdings system

The system compiles and runs on Windows, MacOS, and Linux/Ubuntu. To run the system, one would need to execute the following procedure:

1. Ensure that python3 is installed on the device. Follow the instructions listed here to install Python: https://realpython.com/installing-python/.
2. Open a new terminal (this program will run on MacOS, Windows, or Linux/Ubuntu). The user should ensure the directory they are within is 101139937_4810Assignment.
3. Run the following command to populate the passwd.txt file and see the user permissions:

   python3 UserList.py

4. Run the following command to enter the Finvest Holdings system user interface. This is the primary file that will allow the user to access the system.

   python3 UserInterface.py

The system utilizes a Role-Based Access Control (RBAC) mechanism to assign roles and enforce Finvest Holdings's access control policy. An Excel-based CSV file consists of all the user roles, as well as the permissions that each role would have when accessing the system. The Access Control policy is enforced using AccessControlPolicy.py, which initializes the permissions for each role.

The password file, passwd.txt, consists of four components: the user's username, their role, the salt added to reinforce their password hash, and their hashed password. To hash the password, Python's **hashlib** library was imported, and the hash function SHA-256 was used due to a high level of security and no known vulnerabilities. To add a user to the password file, the PasswordEncrypter.py handles this

functionality and is called by a User entity. The passwd.txt file is re-generated every time UserList.py is ran, which rehashes the passwords and keeps the hashes fresh.

The front-end is handled entirely by UserInterface.py, which is the primary file that is ran when the user intends to access and interact with the system. The user can register an account, which adds a user to the passwd.txt file through the PasswordEncrypter.py file, and they can login and view their permissions. The login feature is also implemented through PasswordEncrypter.py, which iterates through the passwd.txt file, finds the matching username, rehashes the password with the same salt, and compares the hashes to see if they match.

When a user registers an account, their password is validated using a variety of criteria outlined in the contractual obligations. The PasswordChecker.py consists of numerous methods for checking if the password has a suitable length, does not match the user's username, contains the required characters, and does not match a license plate format, telephone number format, or a calendar format. If a user's password is not "strong" enough, they will be notified of the reason and will be redirected to try again.

An example use-case below depicts a process of registering and logging in:

1. Firstly, a user enters the User Interface screen. They must register themselves within the system.

```
Finvest Holdings: Client Records System
Enter L to login, or R to register a new account: R
Entering register...
```

Figure 50: Finvest Holdings General interface, entering the Register interface

2. The user is prompted to enter a role. In this use-case, the User enters "Teller".

```
Finvest Holdings: Client Records System
Register User
First, enter a valid role. List of valid roles include:
Regular Client
Premium Client
Financial Advisor
Financial Planner
Investment Analyst
Compliance Officer
Technical Support
Teller
Role: Teller
```

Figure 51: Example of entering a role into the Register interface

3. The user is then prompted to enter their username and password. Note that the password "Grapefruit5$" is a valid password, and passes all checks. Also note that "Grapefruit5$" is <u>not</u> printed on the user's end and is simply shown for the purpose of this use-case. The password is typically hidden on the user's end.

```
Next, enter a username and password.
Username: maxcurkovic
Enter password:
Grapefruit5$
Account successfully created!
```

Figure 52: Example of a user with username “maxcurkovic” and password “Grapefruit5$” being created

4. As can be seen in the passwd.txt file, the user is successfully registered.

```
caroline Technical Support 1023309634 804dc5379c2f38f58a0184726713e15f874f07b96211d333882c035e74cdccf7
pawel Technical Support 2412850784 299c37238fce3cc15286ee6627ff347de74fa875c9bed86b7f9d46b20ac4e9e2
maxi Teller 175374228 9691112828dfc361b2251a6039a7a06317160288c305347c98c29b82c36c0023
maxcurkovic Teller 2681714053 09a5b76f774c2ea67646f3175c171ce8faad4b8c743776496e8ba794b49c1d77
```

Figure 53: Same user being added to the passwd.txt file

5. The user is redirected back to the main page, and will now begin the login process.

```
Finvest Holdings: Client Records System
Enter L to login, or R to register a new account: l
Entering login...
```

Figure 54: Finvest Holdings General interface, entering the Login interface

6. The user enters their associated username and password. Again, the password will hidden to the user, and is only printed here for this use-case. If the user does not enter the correct username or password, they will receive a message saying such and will be redirected to try again.

```
Finvest Holdings: Client Records System
Login User
Enter your username: maxcurkovic
Enter your password:
Grapefruit5$
```

Figure 55: Example of a user with username “maxcurkovic” and password “Grapefruit5$” being logged in

7. Now that the user is logged in, they will be able to see their username, role, and their permissions associated with the role.

```
Access Granted. Login successful

Username: maxcurkovic
Role: Teller
Permissions:
Account Balance=R
Investments Portfolio=R
Financial Advisor Contact Details=
Investment Analyst Contact Details=
Money Market Instruments=
Consumer Instruments=
Derivatives Trading=
Interest Instruments=
Client Account=
Access Between 9AM-5PM=RW
Validate Modifications=


Type L to logout, any other key to quit program:
```

Figure 56: Example of a user with username “maxcurkovic” and role “Teller” with their permissions

Overall, the Finvest Holdings system addresses all the contractual obligations outlined in the given contract, while talking the client requirements and constraints into consideration. This security system is optimal for the requirements and needs of Finvest Holdings and addresses the desire to have a new user authentication and access control system for proprietary financial software.